Integrating Project Proposal Document

Proposed project: SIMD and multicore acceleration for movie recommendations

Abstract

We propose a movie-to-movie recommendation program using cosine similarity. We'll compare scalar C, single-thread SIMD, and multicore SIMD, including parallel top-K selection. Furthermore, we'll measure when the complete pipeline benefits from these optimizations.

Description

Similarity search is a way to find the items in a collection that are most like a given query. Instead of looking for an exact match, it compares the query with each item and ranks them by similarity.

In our project, we'll represent the query and each item as a vector, which is a list of numbers. We'll compare them using cosine similarity. It measures how closely two vectors point in the same direction; a higher score means the vectors are more similar.

We'll calculate a score between the selected movie and every other movie, then return the K highest-ranking matches. We'll use vectors prepared from movie plot descriptions and exclude the selected movie from its results. This is a content-based movie-to-movie recommendation; it does not use viewing histories or personal profiles.

Deliverables

There are 3 main deliverables we will implement. These are a normal C baseline, SIMD versions with and without multithreading, and movie and synthetic numerical data. We will evaluate their correctness and measure the speedup.

The first deliverable we will implement is the normal C baseline. This program will calculate cosine similarity between a single query vector and multiple candidate vectors sequentially using standard C loops. It can compare one query vector against many vectors and rank the most similar results. This shows how standard scalar code performs without vector hardware, providing our reference point for both ranking and runtime.

The second deliverable we will implement is the SIMD version. Cosine similarity requires dot products and vector magnitude calculations. We'll normalize the candidate vectors once before searching, then use hardware vector registers to compute the dot products across multiple data lanes. We'll handle leftover components using a scalar loop.

We'll also add a multicore version using OpenMP, which divides candidate movies among CPU threads. Each worker will use the SIMD routine and keep its own top-K heap. This parallelizes scoring and selection across workers. Heap updates within a worker will remain sequential. Afterward, we'll merge the local results sequentially and sort the final K matches. Input validation, file loading, movie lookup, and output will also remain sequential.

The third deliverable will be movie and synthetic numerical data. We'll use the CMU Movie Summary Corpus, which contains 42,306 movie plot summaries with associated metadata (Bamman et al., 2013), for the real movie demonstration. We'll prepare plot-description vectors offline using a pretrained text encoder, then load the vectors and movie titles into our C program. Likewise, we'll select and document the encoder and its handling of long plots before implementation and report the usable movie count after cleaning and matching metadata. Synthetic vectors will let us vary candidate count and vector length independently for scaling experiments, which we'll report separately from the movie demonstration. Model training and a website are outside our scope.

The first way of evaluation is by checking output. We'll compare similarity scores with a double-precision reference using a documented numerical tolerance. On identical score arrays, the top-K selection and merge must produce the same ordered movie IDs as a reference full sort. Across arithmetic versions, we'll investigate differences near tied scores because SIMD can change floating-point addition order. We'll also test equal scores, leftover components, uneven worker blocks, and invalid vectors.

Second is measuring speedup. We will measure the execution time of the C, single-thread SIMD, and multicore SIMD versions. We will calculate speedup by dividing the C baseline runtime by the optimized runtime for the same workload. Furthermore, we'll report median times from at least 30 measured runs after warmup. Candidate normalization will be timed separately; complete query timing will include scoring, selection, thread coordination, and merging. Loading, model inference, and printing will stay outside query timing. We'll vary candidate count, vector length, K, and thread count and report slowdowns too.

Target platform

We propose Windows x64, Visual Studio with MSVC, NASM, and an AVX2-capable CPU. The assembly kernels will use 256-bit vector registers, while the C driver will use OpenMP for threading. We'll confirm CPU and operating-system support before implementation and record the actual tool versions and build settings. Benchmarks will use Release builds with automatic vectorization disabled for the scalar reference.

Related work

Pazzani and Billsus (2007) describe content-based recommendation. Sentence-BERT supports representing text as vectors compared through cosine similarity (Reimers & Gurevych, 2019). Faiss discusses both score computation and top-K selection, including CPU SIMD and OpenMP (Douze et al., 2026). Faiss describes CPU threading across queries; our implementation will instead divide the candidate movies for a single query among OpenMP workers, each using SIMD scoring and a local top-K heap, followed by a sequential merge. Johnson et al. (2021) study efficient GPU search and K-selection. Our project will evaluate a CPU implementation of these established ideas; it will not claim a new recommendation algorithm or assume published GPU speedups apply to our hardware.

Mid-project milestones

For Week 10, we'll demonstrate the movie-vector input, a working C baseline, the single-thread SIMD scoring kernel, initial correctness checks, and preliminary timings. We'll also demonstrate candidate partitioning, local top-K heaps, and merging in C. For Week 13, we'll complete SIMD normalization and multicore integration, run the scaling experiments, and deliver the movie demonstration, source code, correctness output, and performance discussion in our public GitHub repository.

References

Bamman, D., O'Connor, B., & Smith, N. A. (2013). Learning latent personas of film characters. In H. Schuetze, P. Fung, & M. Poesio (Eds.), *Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 352-361). Association for Computational Linguistics. https://aclanthology.org/P13-1035/

Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., & Jégou, H. (2026). The Faiss library. *IEEE Transactions on Big Data, 12*(2), 346-361. https://doi.org/10.1109/TBDATA.2025.3618474

Johnson, J., Douze, M., & Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data, 7*(3), 535-547. https://doi.org/10.1109/TBDATA.2019.2921572

Pazzani, M. J., & Billsus, D. (2007). Content-based recommendation systems. In P. Brusilovsky, A. Kobsa, & W. Nejdl (Eds.), *The adaptive web* (pp. 325-341). Springer. https://doi.org/10.1007/978-3-540-72079-9_10

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In K. Inui, J. Jiang, V. Ng, & X. Wan (Eds.), *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (pp. 3982-3992). Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1410